

Abstraction in C# with Real-Time Examples

In this article, I will discuss **Abstraction in C#** with Real-Time Examples. Please read our previous article before proceeding to this article, where we discussed [Encapsulation in C#](#) with Examples. Abstraction in C# is one of the fundamental OOP principles which acts as a supporting principle. That means the Abstraction Principle in C# makes sure that all other three principles (Encapsulation, Polymorphism, and Inheritance) are working together to give the final shape of the project.

What is Abstraction in C#?

The process of representing the essential features without including the background details is called Abstraction. In simple words, we can say that it is a process of defining a class by providing the necessary details to call the object operations (i.e., methods) by hiding its implementation details. It is called abstraction in C#. It means we need to expose what is necessary and compulsory, and we need to hide unnecessary things from the outside world.

Programmatically, we can define abstraction as the process of hiding implementation details of the object's feature and showing only the essential information of the feature to the user. Abstraction lets you focus more on what an object does rather than how it does. That means what are the services available as part of the class that we need to expose, but how are the services implemented that we need to hide? Now, let us try to understand the concept abstraction principle with some real-time examples.

Real-Time Example of Abstraction

To take a real-time example, when we log in to any social networking site like Facebook, Twitter, LinkedIn, etc., we enter our user ID and password, and then we get logged in. Here, we don't know how they are processing the data or what logic or algorithm they are using for login. This information is abstracted/hidden from us since they are not essential to us. This is basically what abstraction is.

Another real-time example is an ATM machine. We all use ATM machines for cash withdrawals, money transfers, retrieving min-statements, etc. in our daily lives. But we don't know internally what things are happening inside an ATM machine when we insert an ATM card for performing different kinds of operations. Information like where the server is, where the database server is, what programming language they use to write the logic, how they are validating the data, how they are implementing logic for various kinds of operations, and what SQL statements get executed on the database when we perform any operations, all these things are hidden from us. What they provide as part of the ATM machine is services (cash withdrawal, money transfer, retrieving min-statement, etc), but how these services are implemented is abstracted to us.

Another real-world example of abstraction could be your TV remote. The remote has different functions like on/off, changing the channel, increasing/decreasing volume, etc. You can use these functionalities by just pressing the button. However, the internal mechanism of these functionalities is abstracted from us as those are not essential for us to know.

Another example of abstraction is when you send an SMS from your mobile, you only type the text and send the message. But you don't know the internal processing or mechanism of the message delivery system. This is nothing but abstraction.

So, in simple words, we can say that abstraction means providing the necessary details to use the services or functionalities but how the services are implemented internally that you need to hide.

Example to Understand Abstraction Principle in C#:

Now, we are going to develop one application to implement the Banking functionality. First, we will develop the application without following the Abstraction Principle, and then we will understand the problems. Then, we will see what are the different mechanisms to implement the abstraction principle in C#. So, what we will do is we will create two classes. One class is for SBI Bank, and another class is for AXIX Bank. As part of each class, we are going to provide 5 services, which are as follows:

1. ValidateCard
2. WithdrawMoney
3. CheckBalance
4. BankTransfer
5. MiniStatement

Then, from the Main method, we will create the instances of each class and will invoke the respective services, i.e., respective methods. Here, you can consider the Main method is the user who will use the services provided by the Bank classes.

```
using System;
namespace GarbageCollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Transaction doing SBI Bank");
            SBI sbi = new SBI();
            sbi.ValidateCard();
            sbi.WithdrawMoney();
            sbi.CheckBalance();
            sbi.BankTransfer();
            sbi.MiniStatement();
            Console.WriteLine("\nTransaction doing AXIX Bank");
            AXIX AXIX = new AXIX();
            AXIX.ValidateCard();
            AXIX.WithdrawMoney();
            AXIX.CheckBalance();
            AXIX.BankTransfer();
            AXIX.MiniStatement();
            Console.Read();
        }
    }

    public class SBI
    {
        public void BankTransfer()
        {
            Console.WriteLine("SBI Bank Bank Transfer");
        }
        public void CheckBalance()
```

```

    {
        Console.WriteLine("SBI Bank Check Balanace");
    }
    public void MiniStatement()
    {
        Console.WriteLine("SBI Bank Mini Statement");
    }
    public void ValidateCard()
    {
        Console.WriteLine("SBI Bank Validate Card");
    }
    public void WithdrawMoney()
    {
        Console.WriteLine("SBI Bank Withdraw Money");
    }
}
public class AXIX
{
    public void BankTransfer()
    {
        Console.WriteLine("AXIX Bank Bank Transfer");
    }
    public void CheckBalanace()
    {
        Console.WriteLine("AXIX Bank Check Balanace");
    }
    public void MiniStatement()
    {
        Console.WriteLine("AXIX Bank Mini Statement");
    }
    public void ValidateCard()
    {
        Console.WriteLine("AXIX Bank Validate Card");
    }
    public void WithdrawMoney()
    {
        Console.WriteLine("AXIX Bank Withdraw Money");
    }
}
}
}

```

Output:

```
Transaction doing SBI Bank
SBI Bank Validate Card
SBI Bank Withdraw Money
SBI Bank Check Balanace
SBI Bank Bank Transfer
SBI Bank Mini Statement
```

```
Transaction doing AXIX Bank
AXIX Bank Validate Card
AXIX Bank Withdraw Money
AXIX Bank Check Balanace
AXIX Bank Bank Transfer
AXIX Bank Mini Statement
```

That is fine. We are getting output as expected. Then what is the problem with the above implementation? The problem is the user of our application accesses the SBI and AXIX classes directly. Directly means they can go to the class definition and see the implementation details of the methods. That is, the user will come to know how the services or methods are implemented. This might cause security issues. We should not expose our implementation details to the outside.

How to Implement Abstraction Principle in C#?

In C#, we can implement the abstraction OOPs principle in two ways. They are as follows:

1. **Using Interface**
2. **Using Abstract Classes and Abstract Methods**

What are [Interfaces](#), and what are [Abstract Methods and Abstract Classes](#) that we will discuss in detail in our upcoming article? But for now, you just need to understand one thing: both interface and abstract classes and abstract methods provide some mechanism to hide the implementation details by only exposing the services. The user only knows what are services or methods available, but the user will not know how these services or methods are implemented. Let us see this with examples.

Example to Implement Abstraction Principle in C# using Interface:

In the below example, I am using an interface to achieve the abstraction principle in C#. Using the interface, we can achieve 100% abstraction. Now, the user will only know the services that are defined in the interface, but how the services are implemented, the user will never know. This is how we can implement abstraction in C# by hiding the implementation details from the user. Here, the user will only know about IBank, but the user will not know about the SBI and AXIX Classes.

```
using System;
namespace GarbageCollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Transaction doing SBI Bank");
        }
    }
}
```

```

        IBank sbi = BankFactory.GetBankObject("SBI");
        sbi.ValidateCard();
        sbi.WithdrawMoney();
        sbi.CheckBalanace();
        sbi.BankTransfer();
        sbi.MiniStatement();
        Console.WriteLine("\nTransaction doing AXIX Bank");
        IBank AXIX = BankFactory.GetBankObject("AXIX");
        AXIX.ValidateCard();
        AXIX.WithdrawMoney();
        AXIX.CheckBalanace();
        AXIX.BankTransfer();
        AXIX.MiniStatement();
        Console.Read();
    }
}

public interface IBank
{
    void ValidateCard();
    void WithdrawMoney();
    void CheckBalanace();
    void BankTransfer();
    void MiniStatement();
}

public class BankFactory
{
    public static IBank GetBankObject(string bankType)
    {
        IBank BankObject = null;
        if (bankType == "SBI")
        {
            BankObject = new SBI();
        }
        else if (bankType == "AXIX")
        {
            BankObject = new AXIX();
        }
        return BankObject;
    }
}

public class SBI : IBank
{
    public void BankTransfer()
    {
        Console.WriteLine("SBI Bank Bank Transfer");
    }
    public void CheckBalanace()
    {
        Console.WriteLine("SBI Bank Check Balanace");
    }
    public void MiniStatement()

```

```

    {
        Console.WriteLine("SBI Bank Mini Statement");
    }
    public void ValidateCard()
    {
        Console.WriteLine("SBI Bank Validate Card");
    }
    public void WithdrawMoney()
    {
        Console.WriteLine("SBI Bank Withdraw Money");
    }
}
public class AXIX : IBank
{
    public void BankTransfer()
    {
        Console.WriteLine("AXIX Bank Bank Transfer");
    }
    public void CheckBalanace()
    {
        Console.WriteLine("AXIX Bank Check Balanace");
    }
    public void MiniStatement()
    {
        Console.WriteLine("AXIX Bank Mini Statement");
    }
    public void ValidateCard()
    {
        Console.WriteLine("AXIX Bank Validate Card");
    }
    public void WithdrawMoney()
    {
        Console.WriteLine("AXIX Bank Withdraw Money");
    }
}
}

```

Output:

```
Transaction doing SBI Bank
SBI Bank Validate Card
SBI Bank Withdraw Money
SBI Bank Check Balanace
SBI Bank Bank Transfer
SBI Bank Mini Statement

Transaction doing AXIX Bank
AXIX Bank Validate Card
AXIX Bank Withdraw Money
AXIX Bank Check Balanace
AXIX Bank Bank Transfer
AXIX Bank Mini Statement
```

Example to Implement Abstraction Principle in C# using Abstract Class and Abstract Methods:

In the below example, we are using abstract class and abstract methods to achieve the abstraction principle in C#. We can achieve 0 to 100% abstraction using the abstract class and abstract methods. In the below example, the user will only know the services that are defined in the abstract class, but how these services are implemented, the user will never know. This is how we can implement abstraction in C# by hiding the implementation details from the user.

```
using System;
namespace GarbageCollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Transaction doing SBI Bank");
            IBank sbi = BankFactory.GetBankObject("SBI");
            sbi.ValidateCard();
            sbi.WithdrawMoney();
            sbi.CheckBalanace();
            sbi.BankTransfer();
            sbi.MiniStatement();
            Console.WriteLine("\nTransaction doing AXIX Bank");
            IBank AXIX = BankFactory.GetBankObject("AXIX");
            AXIX.ValidateCard();
            AXIX.WithdrawMoney();
            AXIX.CheckBalanace();
            AXIX.BankTransfer();
            AXIX.MiniStatement();
            Console.Read();
        }
    }
}
public abstract class IBank
{
```

```

        public abstract void ValidateCard();
        public abstract void WithdrawMoney();
        public abstract void CheckBalanace();
        public abstract void BankTransfer();
        public abstract void MiniStatement();
    }
    public class BankFactory
    {
        public static IBank GetBankObject(string bankType)
        {
            IBank BankObject = null;
            if (bankType == "SBI")
            {
                BankObject = new SBI();
            }
            else if (bankType == "AXIX")
            {
                BankObject = new AXIX();
            }
            return BankObject;
        }
    }
    public class SBI : IBank
    {
        public override void BankTransfer()
        {
            Console.WriteLine("SBI Bank Bank Transfer");
        }
        public override void CheckBalanace()
        {
            Console.WriteLine("SBI Bank Check Balanace");
        }
        public override void MiniStatement()
        {
            Console.WriteLine("SBI Bank Mini Statement");
        }
        public override void ValidateCard()
        {
            Console.WriteLine("SBI Bank Validate Card");
        }
        public override void WithdrawMoney()
        {
            Console.WriteLine("SBI Bank Withdraw Money");
        }
    }
    public class AXIX : IBank
    {
        public override void BankTransfer()
        {
            Console.WriteLine("AXIX Bank Bank Transfer");
        }
    }

```

```

        public override void CheckBalanace()
        {
            Console.WriteLine("AXIX Bank Check Balanace");
        }
        public override void MiniStatement()
        {
            Console.WriteLine("AXIX Bank Mini Statement");
        }
        public override void ValidateCard()
        {
            Console.WriteLine("AXIX Bank Validate Card");
        }
        public override void WithdrawMoney()
        {
            Console.WriteLine("AXIX Bank Withdraw Money");
        }
    }
}

```

Output:

```

Transaction doing SBI Bank
SBI Bank Validate Card
SBI Bank Withdraw Money
SBI Bank Check Balanace
SBI Bank Bank Transfer
SBI Bank Mini Statement

Transaction doing AXIX Bank
AXIX Bank Validate Card
AXIX Bank Withdraw Money
AXIX Bank Check Balanace
AXIX Bank Bank Transfer
AXIX Bank Mini Statement

```

Note: Using abstract class, we can achieve 0 to 100% abstraction. The reason is that you can also provide implementation to the methods inside the abstract class. It does not matter whether you implement all methods or none of the methods inside the abstract class. This is allowed, which is not possible with an interface.

Encapsulation vs Abstraction in C#

1. The Encapsulation Principle is all about data hiding (or information hiding). On the other hand, the Abstraction Principle is all about detailed hiding (implementation hiding).
2. Using the Encapsulation principle, we can protect our data, i.e., from outside the class, nobody can access the data directly. We are exposing the data through publicly exposed methods and properties. The advantage is that we can validate the data before storing and returning it. On the other hand, using the Abstraction Principle, we are exposing only the services so that the user can consume the

services, but how the services/methods are implemented is hidden from the user. The user will never know how the method is implemented.

3. With the Encapsulation Principle, we group data members and member functions into a single unit called class, interface, enum, etc. On the other hand, with the Abstraction Principle, we are exposing the interface or abstract class to the user and hiding implementation details, i.e., hiding the child class information.
4. We can implement Encapsulation by declaring the data members as private and exposing the data members only through publicly exposed methods and properties with proper validation. On the other hand, we can implement abstraction through abstract classes and interfaces.
5. In abstraction, only the abstract view is presented to the user, while complex and detailed data is hidden from the user. On the other hand, in encapsulation, data member and member functions are bundled as a single unit and can be protected or made accessible using access modifiers and getter and setter methods.
6. Abstraction in C# is used to hide unwanted data and shows only the required properties and methods to the user. Encapsulation in C# is used to bind data members and member functions into a single unit to prevent outsiders from accessing it directly.

Advantages of Abstraction Principle in C#

1. The Abstraction Principle reduces the complexity of viewing things. It only provides the method signature by hiding how the method is actually implemented.
2. The Abstraction Principle helps to increase the security of an application or program as we are only providing the necessary details to call the method by hiding how the methods are actually implemented.
3. With the Abstraction Principle, the enhancement will become very easy because without affecting end-users, we can able to perform any type of changes in our internal system.
4. Without the Abstraction principle, maintaining application code is very complex. Abstraction gives one structure to program code.

In the next article, I am going to discuss [Inheritance in C#](#) with Examples. Here, in this article, I try to explain the **Abstraction in C#** with Examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, questions, or comments about this Abstraction in C# with the Examples article.